

U3. AP3. Cliente-Servidor TCP con Eco

1. Eco binario interactivo

Objetivo

Practicar:

- ServerSocket y Socket
- Streams **de bytes** (InputStream / OutputStream)
- Detección del cierre de conexión
- Comunicación **un cliente a la vez**

Enunciado

Implementa dos programas en Java:

1. Servidor TCP

- Escucha en un puerto (por ejemplo, 5000).
- Acepta **un solo cliente**.
- Lee datos del cliente usando un **InputStream de bytes**.
- Por cada bloque de bytes recibido:
 - Los reenvía tal cual al cliente (comportamiento “eco”).
- Cuando el cliente cierra la conexión:
 - Muestra un mensaje por consola.
 - Cierra el socket del cliente.
 - El servidor termina.

Importante:

No usar BufferedReader, PrintWriter, DataInputStream, etc.

Solo streams de bytes.

Cliente TCP

- Se conecta al servidor.
- Pide mensajes al usuario
- Convierte los mensajes a bytes y los envía
- Después de cada envío lee la respuesta del servidor y la muestra por pantalla.
- Cuando el usuario escribe un punto (.) el cliente no lo envía sino que cierra el socket

Restricciones técnicas

- Comunicación mediante:
 - InputStream

- OutputStream
- Lectura usando:


```
byte[] buffer = new byte[1024];
int bytesLeidos = in.read(buffer);
```
- El fin de comunicación se detecta cuando read() devuelve -1.

Consejos

- Al escribir bytes, write() no garantiza el envío inmediato. Hay que usar flush() explícito para forzar la salida de los bytes.
- La conversión a String se puede realizar con:


```
new String(buffer, 0, bytesLeidos)
```
- La conversión de String al array de bytes se puede realizar con:


```
byte[] datos = (línea + "\n").getBytes();
```

Pero no hay que olvidar que la salida sigue siendo binaria.

2. Eco texto interactivo

Partiendo de la solución al ejercicio anterior, ahora se pide implementar la misma aplicación Cliente–Servidor “Eco Interactivo”, pero utilizando comunicación en **modo texto**.

En esta nueva versión:

- La comunicación se realizará mediante **líneas de texto**
- El servidor leerá los mensajes usando readLine()
- Cuando el usuario escriba un **punto (.)**:
 - el cliente cerrará la conexión
 - el servidor detectará el cierre mediante readLine() == null

Restricciones

Se deben usar:

- BufferedReader
- PrintWriter (o BufferedWriter)

No se permite mezclar streams de bytes **y** texto

Consejo

Crear PrintWriter con autoFlush = true, así:

```
new PrintWriter(socket.getOutputStream(), true);
```